WHITEWRITE

CHARACTER FORGE

Технічне завдання на створення AI-майстерні персонажів та екіпірування

Параметр	Значення
Продукт	WhiteWrite · Game Bible
Модуль	Character Forge / Character & Equipment Workshop
Мета	Покрокове виробництво персонажів, 3D-моделей та сумісного екіпірування
Адресат	Claude Code / команда розробки WhiteWrite
Версія	1.0 · 02.11.2026

Ключовий принцип: WhiteWrite має бути не просто генератором зображень або сховищем файлів, а системою, яка веде розробника від концепції героя до перевірених і готових до інтеграції ігрових асетів.

Перед внесенням змін Claude Code повинен провести аудит актуального репозиторію та адаптувати рішення до реального коду. Цей документ описує цільову поведінку й архітектуру, а не стверджує, що всі наведені сутності вже існують.

1. Мета та бачення

Створити в межах WhiteWrite окрему майстерню Character Forge, інтегровану з Game Bible. Вона організовує повний виробничий цикл персонажа та його обладунків, зберігає контекст, файли, версії, рішення й статуси.

- Єдиний контекст: раса, клас, лор, характеристики, художній стиль, референси та технічні обмеження доступні на кожному етапі.
- Покрокове виробництво: кожен етап має вхідні дані, інструменти, результати, статус, критерії приймання та наступні кроки.
- Людина затверджує результат: AI пропонує й генерує, але не замінює затверджені версії автоматично.
- Провайдер-незалежність: Meshy — один із адаптерів, а не жорстка залежність усієї системи.
- Розширюваність: згодом підтримати монстрів, NPC, зброю, предмети, тварин, маунтів та environment assets.

Не входить у перший MVP: повноцінний 3D-редактор у браузері, автоматичне виправлення складної топології, автоматичний rigging усіх типів істот та пряме редагування Blender-сцен.

2. Інтеграція з наявним WhiteWrite

Попередній огляд репозиторію вказує на наявність Game Bible, World Canon, візуального Style Check, файлового зберігання, production planning, Firebase/Cloud Functions та GitHub/Claude Code інтеграцій. Усе це необхідно повторно перевірити в актуальній гілці.

Наявна зона	Очікуване використання
Game Bible / World Canon	Канонічні дані персонажа, рас, фракцій, лору й термінології.
Visual Style / Style Check	Перевірка концептів та асетів на відповідність стилю; не дублювати наявний механізм.
Файлове сховище	Зберігання зображень, 3D-файлів, текстур, прев’ю та експортів.
Production Plan	Планування задач, залежностей, етапів і статусів, якщо наявний механізм підходить.
Firebase / Cloud Functions	Серверна оркестрація генерацій, збереження метаданих, доступи та облік витрат.
GitHub / Claude Code	Передача затверджених технічних специфікацій у робочий репозиторій.

Обов’язкова інструкція: спочатку перевірити актуальну структуру папок, entry points, маршрутизацію, дизайн-систему, Firestore/Storage схеми, Cloud Functions та security rules. Не створювати паралельну архітектуру й не дублювати існуючі модулі. Якщо попередні висновки не відповідають коду — орієнтуватися на код.

3. Інформаційна архітектура та UX

У Game Bible додати розділ Character Forge. Він відкриває список персонажів конкретної гри та окрему робочу область кожного героя.

- Персонажі: список, пошук, фільтри, створення, архівація, статуси й прогрес.
- Екіпірування: каталог предметів, типи слотів, прив’язка до персонажа, сумісність.
- Бібліотека асетів: повторно використовувані моделі, текстури, матеріали, анімації та референси.
- Виробництво: задачі, залежності, погодження, помилки та історія.
- Робоча область героя: Профіль; Моделі та частини тіла; Екіпірування; Анімації; Виробництво та версії.

Інтерфейс має бути візуальним: велике прев’ю героя, прогрес виробництва, список етапів, картки асетів, панель властивостей та дії «Згенерувати», «Завантажити», «Порівняти», «Затвердити», «Повернути версію». Розширені технічні параметри відкривати за потреби.

4. Виробничий процес персонажа

Процес — керований workflow із залежностями. Користувач може повернутися до попереднього етапу, створити нову версію або змінити рішення. Затверджені результати не перезаписуються автоматично.

Етап	Зміст / результати	Критерій переходу
1. Character Brief	Раса, клас, роль, лор, силует, пропорції, референси, палітра.	Brief заповнений і затверджений.
2. Concept Art	Концепти, фронт/бік/спина, T/A-Pose reference, обличчя, одяг.	Фінальний дизайн затверджено.
3. Base Mesh	3D-база, геометрія, пропорції, UV, матеріали, вихідний файл і прев’ю.	Модель перевірена й затверджена.
4. Rigging	Скелет, skin weights, rig profile, тест деформацій.	Rig затверджений; визначено skeleton profile.
5. Equipment	Слоти, предмети, концепти, 3D-моделі, матеріали, варіанти.	Предмет має специфікацію та слот.
6. Integration	Підгонка, attachment points, body masking, clipping/rig tests.	Перевірки пройдені або є погоджений виняток.
7. Animation	Idle, walk/run, attack, block/dodge, death та потрібні кліпи.	Анімації перевірені на моделі й екіпіруванні.
8. Export	FBX/GLB, текстури, матеріали, LOD, prefab package.	Експортний пакет зібраний і перевірений.

Workflow має налаштовуватися для різних типів персонажів. Бос, NPC і звичайний герой можуть мати різні вимоги. Не зашивати один pipeline як незмінний для всіх.

5. Equipment Studio — модульна система екіпірування

Базове тіло персонажа — окремий стабільний asset. Екіпірування — окремі моделі, сумісні з rig/body profile, а не щоразу перегенеровані частини тіла.

Приклади слотів: Head, Hair, Face, Shoulders, Chest, Arms, Hands, Waist, Legs, Feet, Back/Cloak, WeaponMainHand, WeaponOffHand, Accessory. Список слотів налаштовується під гру.

- Предмет має тип, слот, назву, опис, матеріали, варіанти, іконку, концепт, 3D-файл та статус.
- Предмет прив’язується до персонажа або спільного Body/Rig Profile для повторного використання.
- Підтримати Loadouts, варіанти одного предмета та альтернативні скіни.
- Підтримати body hiding/masking: приховувати стопу під чоботом або ділянки тулуба під бронею.
- Підтримати attachment points для зброї, щитів, плащів, аксесуарів.
- Skinned equipment використовує сумісний skeleton; жорсткі предмети кріпляться до кістки/точки.

Auto Split або локальне AI-редагування не гарантує production-ready топологію, UV, коректні ваги чи rig compatibility. Результат має проходити перевірку у Blender або іншому DCC-процесі.

6. AI Prompt Builder та провайдери

WhiteWrite збирає контекст персонажа й формує промпт та параметри для конкретної операції. Користувачеві не потрібно вручну копіювати опис героя.

Контекст	Що передається
Character Context	Раса, клас, силует, пропорції, роль, затверджений дизайн.
Art Direction	Художній стиль, палітра, матеріали, референси, обмеження.
Asset Brief	Тип предмета, слот, функція, форма, деталі, деталізація.
Technical Profile	Target engine, format, scale, polycount, texture resolution, rig/body profile.
References	Затверджені зображення, попередні моделі та сумісні предмети.

Provider Adapter: інтерфейс приховує специфіку Meshy та інших сервісів. Орієнтовні операції: generateConcept, imageTo3D, textTo3DPreview, refineTexture, remesh, rig, animate, getTaskStatus, cancelTask, downloadResult. Реалізовувати лише реально підтримувані конкретним провайдером операції.

- Meshy — перший кандидат на 3D provider; перевіряти актуальні API endpoints, моделі, тарифи й ліміти.
- Text-to-3D може бути двоетапним Preview → Refine; UI показує обидва етапи та витрати.
- Image-to-3D використовує затверджений концепт; зберігати task ID, параметри, статус і результат.
- Manual Import — повноцінний шлях для моделей, створених поза WhiteWrite.
- API keys зберігати тільки на сервері/у secret storage, ніколи не віддавати браузеру.

Офіційна документація для перевірки під час реалізації: https://docs.meshy.ai/en/api/image-to-3d та https://docs.meshy.ai/en/api/text-to-3d. API може змінюватися; не використовувати застарілі параметри без звірки.

7. Compatibility & Validation

Система зберігає технічний паспорт кожного асета та показує зрозумілі результати перевірок. Не заявляти автоматичну перевірку того, для чого немає реального механізму.

Поле	Призначення
Asset Type / Slot	Тип предмета й слот, наприклад Boots / Feet.
Rig Profile	Ідентифікатор сумісного rig.
Body Profile	Профіль пропорцій тіла, наприклад Female_Elf_A.
Skeleton Reference	Посилання на затверджений skeleton/rig.
Scale / Units	Масштаб і одиниці вимірювання.
Format	FBX, GLB та інші дозволені формати.
Materials / Textures	Base Color, Normal, Metallic, Roughness, Emission — де застосовно.
LOD / Polycount	Цільові й фактичні показники, якщо доступні.
Body Mask	Ділянки базового тіла, які приховуються під предметом.
Attachment Points	Кістки та точки кріплення.
Validation Status	Pending, Passed, Warning, Failed, Manually Approved.

- Перевіряти слот, rig/body profile, формат, розмір файлу та обов’язкові метадані.
- За наявності інструментів перевіряти skeleton compatibility, матеріали, LOD і полігональність.
- Clipping і деформації перевіряти через DCC/Unity workflow; не імітувати цю перевірку лише UI.
- Дозволити ручне затвердження з коментарем, автором, датою та посиланням на версію.

- Warnings відокремлювати від hard failures; пояснювати, що блокує перехід.

8. Цільова модель даних

Структура концептуальна. Адаптувати її до чинної схеми Firestore та naming conventions, не дублювати існуючі сутності.

```
worlds/{worldId}/games/{gameId}/ characters/{characterId} profile, design, rig, bodyProfile models/{modelId}
revisions/{revisionId} bodyParts/{partId} equipment/{equipmentId} definition, compatibility
revisions/{revisionId} animations/{animationId} productionTasks/{taskId} approvals/{approvalId}
assetLibrary/{assetId} productionJobs/{jobId} providerTasks/{taskId}
```

- Firestore — метадані, зв’язки, стани, історія, задачі та права доступу.
- Firebase Storage або наявне object storage — GLB/FBX, текстури, зображення та архіви.
- Не зберігати великі бінарні файли у Firestore.
- Кожна генерація — окремий job: provider, providerTaskId, input snapshot, status, progress, error, cost estimate, timestamps, output references.
- Зберігати snapshot контексту, реально використаного під час генерації.
- Версії immutable: новий результат створює revision, не перезаписує затверджений файл.

9. Backend, черги, безпека та витрати

- Frontend викликає серверну функцію створення job; сервер перевіряє користувача, workspace/game permissions, entitlement та баланс.
- Backend створює job, записує input snapshot, викликає провайдера та зберігає provider task ID.
- Статус оновлюється webhook-ом, якщо підтримується, або контрольованим polling із backoff. Polling не повинен створювати нову платну задачу.
- Обробити queued, processing, succeeded, failed, cancelled, timeout та rate-limit стани.
- Використовувати idempotency keys або еквівалентний захист від подвійного запуску.
- Показувати оцінку вартості до запуску, фактичне списання, провайдера та використані кредити.
- Додати rate limits, квоти, ліміти паралельності, retry policy та audit log.
- Firestore/Storage rules мають гарантувати доступ тільки до дозволених світів/ігор.
- Перевіряти MIME type, розмір, розширення, власника та безпечність файлів.

Для платних AI-операцій обов’язкові підтвердження запуску, зрозуміла оцінка вартості, захист від повторного списання та журнал транзакцій. Секрети провайдерів — тільки server-side.

10. План реалізації та Definition of Done

Фаза	Реалізація	Критерій готовності
1. Character Manager	Навігація, список персонажів, профіль, референси, етапи, Firestore.	Створення/редагування/повторне відкриття працює; доступи перевірені.
2. Character Art Studio	Галерея концептів, варіанти, approval, Style Check.	Концепт прив’язаний до героя; є історія версій.
3. 3D Production	Manual import, Meshy provider, job lifecycle, preview/refine.	Задачу можна запустити, відстежити й зберегти; помилки/витрати видимі.
4. Equipment Studio	Слоти, каталог, предмети, прив’язка, loadouts.	Предмет створюється, призначається слоту, має кілька версій.
5. Compatibility	Rig/body profiles, attachment points, body masks, validation.	Статус сумісності пояснений; hard failures блокують approval.
6. Animation & Export	Анімації, export manifest, FBX/GLB, текстури, LOD, Unity handoff.	Експортний пакет містить потрібні файли й метадані.

11. Рекомендований MVP

- Створення персонажа з контекстом Game Bible.
- Виробничі етапи та прогрес.
- Завантаження й затвердження концепт-арту.
- Ручне завантаження базової 3D-моделі (GLB/FBX).
- Каталог слотів екіпірування та створення предметів.
- Прив’язка предмета до персонажа та compatibility metadata.
- Історія версій, approval status і коментарі.
- Підготувати provider abstraction, але не робити платну Meshy-генерацію блокером першого MVP.

Спочатку довести workflow і модель даних на ручному імпорті. Потім додати Meshy. Так UX/дані не змішуються з ризиками зовнішнього API та платних генерацій.

12. Тестування та нефункціональні вимоги

- Unit tests для статусів workflow, прав доступу, слотів і переходів.
- Integration tests для Firestore/Storage rules та Cloud Functions.
- Mock provider tests: success, failure, timeout, rate limit, повторний webhook, duplicate submit.
- Перевірити ізоляцію даних між workspace/game.
- Перевірити відновлення стану після reload та повторного входу.
- Перевірити desktop/mobile адаптивність.
- Затверджені версії не втрачаються після невдалої нової генерації.
- Логи не містять API keys, приватних URL або чутливих даних.

13. Інструкції Claude Code перед початком

- Спочатку проаналізуй актуальний репозиторій: папки, entry points, routing, дизайн-систему, Firebase schema, Cloud Functions, security rules і Game Bible.
- Підготуй короткий audit report: що повторно використати, що розширити, ризики та файли для змін.
- Не переписуй Game Bible і не дублюй Style Check, file manager, project planning або Firebase abstractions.
- Працюй фазами. Перед кожною фазою визначай файли/модулі; після — запускай build, lint і тести.
- Не роби destructive migrations; нові поля/колекції мають бути сумісні зі старими даними.
- Не вигадуй API endpoints чи capabilities. Перевіряй актуальну документацію.
- Якщо потрібен Blender/Unity worker для rigging, topology чи clipping validation — спроєктуй інтеграційний контракт, не імітуй перевірку в UI.
- Новий UI має відповідати існуючому стилю WhiteWrite.

14. Очікувані результати від Claude Code

- Architecture audit і план змін у репозиторії.
- UX-потоки та перелік екранів/станів.
- Схема даних і правила доступу.
- План фаз із залежностями та ризиками.
- Реалізація MVP Character Manager та Equipment foundations.
- Тести, build results і список відомих обмежень.
- Backlog для Meshy, compatibility validation, rigging/animation та Unity export.

15. Основні терміни

Термін	Значення
Character Brief	Канонічний опис персонажа та його вимоги.
Base Mesh	Базова геометрія персонажа.
Rig / Skeleton	Скелет і система деформації.
Equipment Slot	Логічне місце для предмета.
Skinned Mesh	Геометрія, прив’язана до скелета через ваги.
Attachment	Предмет, закріплений на кістці/точці.
Body Mask	Правило приховування частин тіла під екіпіруванням.
Revision	Незмінна версія результату або файлу.
Provider Job	Асинхронна задача зовнішнього AI-провайдера.
Approval	Явне прийняття конкретної версії користувачем.

Кінцеве бачення

Character Forge має перетворити розробку персонажа з хаотичного набору промптів, файлів і ручних операцій на прозорий, повторюваний та керований виробничий процес. Спочатку — персонажі й екіпірування; надалі — універсальна AI Asset Production Studio для всіх категорій ігрового контенту.

Документ підготовлено як продуктовий і технічний бриф для Claude Code. Остаточні рішення щодо структури коду, колекцій, маршрутів і компонентів приймати після аудиту актуального репозиторію.